Chapter 5
Query Optimization

How the DBMS chooses one plan among thousands without running any of them, and why it sometimes chooses wrong

Advanced Data Management

Based on the Chapter 5 lecture and Lab 5

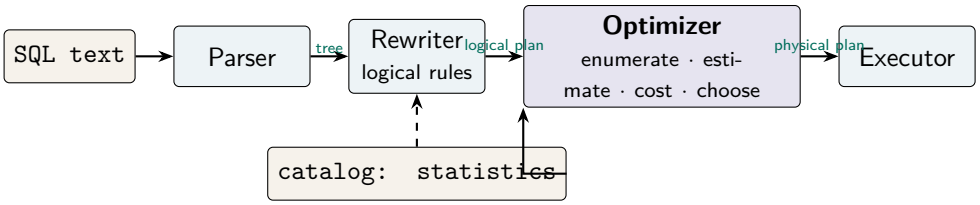
How to read this chapter

Chapter 4 built the toolbox: scans, external sort, three join algorithms, two kinds of aggregation, each with a cost formula in pages. It ended on a problem: for one query there are many correct plans, their costs differ by orders of magnitude, and nobody in the toolbox decides. This chapter is about the decider. The optimizer rewrites the query into cheaper equivalent forms, guesses how many rows each operator will see, prices every candidate plan with the Chapter 4 formulas, searches the join orders, and hands the cheapest plan to the executor. Every one of those steps can be inspected in PostgreSQL, and the last sections show how to read what it did and how to repair it when the guess was wrong. By the end you should be able to estimate a selectivity and a join size by hand, explain why a wrong estimate at a leaf ruins the whole plan, run the System-R dynamic programme on a small example, and diagnose the two classic failures, stale statistics and correlated columns, from an `EXPLAIN ANALYZE`.

1 The Optimizer's Job

1.1 Where it sits

Chapter 1 traced a query through parser, rewriter, optimizer and executor. The optimizer is the stage between the **logical plan** (what to compute: a tree of relational operators) and the **physical plan** (how to compute it: which scan, which join algorithm, in which order).



The one constraint that shapes everything: the optimizer must choose *without running* anything. It has only the **catalog**, the system tables that describe the schema, the indexes and the statistics collected by `ANALYZE`. Every decision below is a prediction made from those numbers.

1.2 Two kinds of optimization

Concept: Logical versus physical

Logical optimization rewrites the query into an equivalent but cheaper form using identities of relational algebra: filter before joining, drop unused columns early, turn a subquery into a join. The result is still a logical plan, independent of how any operator runs.
Physical optimization picks the concrete operators: sequential scan or index scan for each table, nested-loop, sort-merge or hash for each join, and the order in which the joins happen. It needs costs, and costs need estimates of sizes.

The two are not independent. A rewrite that shrinks an input changes the physical choice (a 2,000-row input fits a hash table; a 2,000,000-row one may not), and the physical choice is what makes the rewrite worth anything. Real optimizers interleave them.

1.3 Why it is hard

Three facts, all consequences of Chapter 4:

- **Many plans are correct.** For a join of n tables there are $n!$ left-deep orders, and for each order every join can run three ways and every table can be read two ways.
- **Their costs differ enormously.** The lecture's nightly dashboard ran for 45 minutes with a nested-loop join and 20 seconds with a hash join on the same data: a 135× gap between two plans that both compute the right answer.
- **The inputs to the cost formulas are guesses.** B_R and $|R|$ for a base table are in the catalog; for the output of a filter or a join they must be *estimated*, and a wrong estimate prices every plan above it wrongly.

Key insight. The optimizer is a predictor, not a measurer. It is only as good as its statistics, and the two classic failures of this chapter are both failures of statistics, not of search.

Summary

The optimizer turns a logical plan into a physical plan without running the query, using catalog statistics. Logical rewrites keep the answer and shrink the work; physical choices pick scans, join algorithms and join order by estimated cost. Correct plans differ in cost by orders of magnitude, so the choice matters, and the estimates it rests on are the weak point.

2 Logical Rewrites

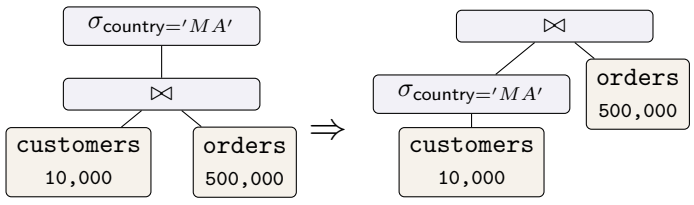
2.1 The algebra behind SQL

A `SELECT` compiles to relational algebra: σ_p (selection, keep rows satisfying p), π_A (projection, keep columns A), $\bowtie$ (join), γ (grouping). These operators obey identities, and every identity is a licence to rewrite a plan into another one with the same answer. The ones that matter:

Identity	Reading
$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ when p mentions only R	**Selection pushdown**: filter before joining.
$\sigma_{p \wedge q}(R) = \sigma_p(\sigma_q(R))$	Split a conjunction; push each part as far down as it can go.
$\pi_A(R \bowtie S) = \pi_A(\pi_{A \cup J}(R) \bowtie \pi_{A \cup J}(S))$, J the join columns	**Projection pushdown**: drop unused columns early.
$R \bowtie S = S \bowtie R$	Join is **commutative**: either side may be build or outer.
$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$	Join is **associative**: any order of joins is legal.

Commutativity and associativity together say: *the join order is free*. That single fact creates the search space of Section 6.

2.2 Selection pushdown



join sees 10,000 × 500,000 input rows, emits 500,000, then filters

filter first: 2,000 customers enter the join, which emits 100,000

The filter mentions only `customers`, so it may move below the join. Same answer, and the expensive operator now sees one fifth of the rows on one side. In PostgreSQL this shows up as a `Filter:` line on the `Seq Scan` *under* the join rather than above it (Section 7.4). Chapter 1 called this the single most

important rewrite; it is applied by every optimizer, always, because it never costs anything and almost always pays.

2.3 The other rewrites

- **Projection pushdown.** If the query only needs `name` and `amount`, the join need not carry the other twelve columns through its hash table and output. Narrower rows mean fewer pages per intermediate result (Chapter 2: rows per page is what turns row counts into page counts).
- **Subquery flattening.** `WHERE cust_id IN (SELECT id FROM customers WHERE country='MA')` becomes a join (a semi-join) between `orders` and the filtered `customers`. A nested subquery executed per outer row is a disguised nested loop; as a join it can use hash or merge.
- **Constant folding and dead predicates.** `WHERE 1 = 1` disappears. The condition `x > 5 AND x > 3` collapses to `x > 5`. A contradiction such as `a = 5 AND a = 6` makes the branch empty, and the optimizer may delete the scan.
- **View expansion.** A view is a stored query; its definition is pasted in so the rewrites above can cross the view boundary.

All of these are **answer-preserving**: they never change the result, so the optimizer applies them unconditionally. What they produce is a set of candidate *logical* plans; choosing among them is the cost model's job.

Worked example: The lecture query, rewritten

```
SELECT c.name, sum(o.amount)
FROM customers c JOIN orders o ON o.cust_id = c.id
WHERE c.country = 'MA' GROUP BY c.name;
```

Logical plan after rewriting:

$$\gamma_{\text{name, sum(amount)}}\Big(\pi_{\text{name,amount}}\big(\,\sigma_{\text{country}='MA'}(\text{customers})$$
$$\bowtie_{\text{id}=\text{cust_id}}\ \pi_{\text{cust_id,amount}}(\text{orders})\,\big)\Big).$$

The filter went under the join (selection pushdown); only three columns cross the join (projection pushdown). Nothing physical has been decided yet: the join could still be any of the three algorithms.

Summary

Relational algebra identities let the optimizer move filters down, drop columns early, flatten subqueries and reorder joins without changing the answer. Selection pushdown is the one that matters most and is always applied; commutativity and associativity of joins are what make join ordering a search problem.

3 Statistics: What the Optimizer Knows

3.1 The catalog's numbers

`ANALYZE` samples each table (by default 300 × `default_statistics_target` = 30,000 rows) and stores, per table and per column, the summaries the estimator needs.

Statistic	Where	What it answers
row count, page count	`pg_class`: `reltuples`, `relpages`	$\|R\|$ and B_R for every base table
`n_distinct`	`pg_stats`	how many different values: selectivity of `col = v` is about 1/n_distinct
`null_frac`	`pg_stats`	fraction of NULLs; they satisfy no comparison
most common values (MCV) + frequencies	`pg_stats`	exact selectivity for the heavy hitters; captures skew
histogram bounds	`pg_stats`	selectivity of ranges (`<`, `BETWEEN`) over the non-MCV values
correlation	`pg_stats`	how well physical order follows the column: index-scan cost
extended statistics	`pg_statistic_ext`	dependencies and joint distinct counts across columns (Section 7)

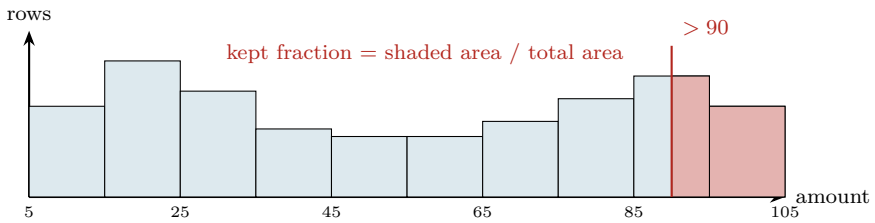
```
SELECT attname, n_distinct, null_frac, most_common_vals,
    most_common_freqs
FROM pg_stats WHERE tablename = 'customers' AND attname = 'country';
--  country | 5 | 0 | {MA,FR,ES,US,DE} | {0.2,0.2,0.2,0.2,0.2}
```

Five countries, each a fifth of the rows: the planner will give `country = 'MA'` selectivity 0.2 and estimate $0.2 \times 10{,}000 = 2{,}000$ customers. That is the raw material of every cost below.

In the real world. `n_distinct` is stored negative when it scales with the table: -1 means "every row is distinct" (a key), -0.5 means "half as many distinct values as rows".

3.2 Histograms

For a range predicate the optimizer needs the *distribution*, not just the count of distinct values. PostgreSQL keeps an **equi-depth histogram**: k bucket boundaries chosen so that each bucket holds the same number of rows (by default 100 buckets, so each holds 1% of the non-MCV rows).



(The bars are drawn with varying heights to show a skewed column; in an equi-depth histogram the *areas* are equal and the bucket *widths* vary. Either way the estimate is the same rule.)

Mechanism: Range selectivity from an equi-depth histogram

Each of the k buckets holds $1/k$ of the rows. For `col > v`: count the buckets entirely above v, each contributing $1/k$; for the bucket containing v, assume values are uniform inside it and add the fraction of that bucket above v. For `BETWEEN`, subtract two such estimates.

Worked example: `amount > 90` on the lab's orders

Amounts are uniform on 5–105, so ten equi-depth buckets have bounds $5, 15, 25, \ldots, 105$. The value 90 falls in the bucket 85–95, half way. Buckets entirely above: one (95–105), contributing 0.1. The partial bucket contributes $0.5 \times 0.1 = 0.05$. Selectivity 0.15, estimated rows $0.15 \times 500{,}000 = 75{,}000$. The lecture's example plan shows `rows=5000` for the same predicate, which corresponds to a 1% threshold such as `amount > 104`; the deck's figures are illustrative, and your own `EXPLAIN` will give the histogram's answer for your data. The method is what the exam tests.

Summary

`ANALYZE` stores row and page counts, `n_distinct`, null fraction, the most common values with their frequencies, and an equi-depth histogram per column. Equality uses `n_distinct` or an MCV frequency; ranges use the histogram. Everything the optimizer predicts is derived from these numbers, and they are only as fresh as the last `ANALYZE`.

4 Cardinality Estimation

4.1 Selectivity as probability

Concept: Selectivity

The **selectivity** of a predicate p on a relation R is the fraction of rows that satisfy it, $\text{sel}(p) = |\sigma_p(R)|/|R|$, a number in $[0, 1]$. Read it as the probability that a random row passes. The estimated output of a filter is then $|R| \cdot \text{sel}(p)$, and the estimated page count is that divided by rows per page.

The rules, each a probability rule in disguise:

Predicate	Estimate	Source
`col = v`	1/n_distinct, or the MCV frequency of v if listed	uniformity, or the MCV list
`col > v`, `BETWEEN`	fraction of the histogram beyond v	histogram
`col IS NULL`	`null_frac`	stored
$p \wedge q$	$\text{sel}(p) \cdot \text{sel}(q)$	**independence assumption**
$p \vee q$	$\text{sel}(p) + \text{sel}(q) - \text{sel}(p)\,\text{sel}(q)$	inclusion–exclusion, with independence
$\neg p$	$1 - \text{sel}(p)$	complement
`col = $1` (unknown constant)	1/n_distinct (equality), a default such as 1/3 for ranges	no better information

Worked example: The lecture exercise

A table has 1,000,000 rows; `country = 'MA'` has selectivity 0.2 and `status = 'active'` 0.1. Rows for `country = 'MA'`: $0.2 \times 10^6 = 200{,}000$. Rows for the AND, assuming independence: $0.2 \times 0.1 \times 10^6 = 20{,}000$. If in fact every active user is Moroccan, then *active* implies *MA* and the conjunction is just `status = 'active'`: 100,000 rows. The independence estimate is five times too **low**. Correlation makes conjunctions less selective than their product, and the error grows with the strength of the dependency.

4.2 Join size

Mechanism: Estimating an equi-join

For $R \bowtie_{R.a=S.b} S$ with d_R distinct values of a and d_S distinct values of b:

$$|R \bowtie S| \;\approx\; \frac{|R| \cdot |S|}{\max(d_R, d_S)}.$$

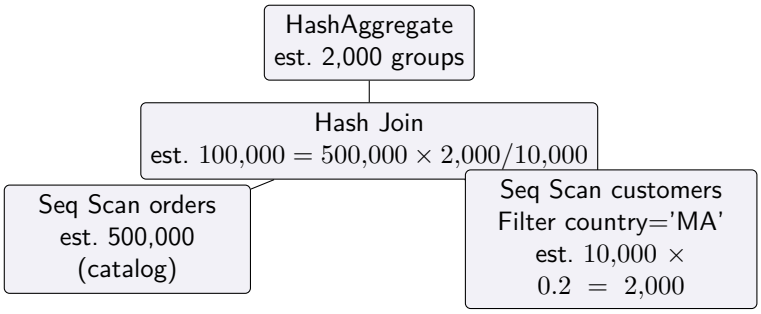
Reasoning: assume the side with more distinct values contains every value of the other side (the *containment* assumption). Each row of S then matches, on average, $|R|/d_R$ rows of R when $d_R \geq d_S$, giving $|S| \cdot |R|/d_R$; symmetrically otherwise. Picking the max covers both cases.

Two instances you should recognise on sight:

- **Key–foreign key join.** $R.a$ is a primary key, so $d_R = |R|$: the estimate is $|R| \cdot |S|/|R| = |S|$. Every order has exactly one customer, so customers $\bowtie$ orders has as many rows as orders: 500,000. If customers was first filtered to a fifth, the join has about a fifth of the orders, 100,000, because the filter removes a fifth of the key values and with them a fifth of the matching orders.
- **No join predicate.** A cross product: $|R| \cdot |S|$, the thing join ordering must avoid.

4.3 Estimates propagate upward

Every operator's estimate is computed from its children's estimates. A plan is priced bottom-up, so the leaves feed everything.



Now suppose the leaf estimate is wrong by a factor of ten (2,000 estimated, 20,000 real, because the statistics are stale). The join estimate is wrong by the same factor, the hash table is ten times larger than planned and may spill, and worse: the optimizer may have chosen a nested loop *because* it believed the outer side had 2,000 rows. Errors do not cancel; through joins they multiply.

Key insight. Cardinality estimation is the hardest part of optimization because every later decision, join algorithm, join order and access path, is a function of the estimated row counts, and an error at a leaf is amplified at every join above it.

Summary

Selectivity is the fraction of rows a predicate keeps; equality uses 1/n_distinct or MCV frequencies, ranges use the histogram, AND multiplies (independence), OR adds and subtracts the product. An equi-join is about $|R||S|/\max(d_R, d_S)$, which collapses to $|S|$ for a key–foreign key join. Estimates are computed bottom-up, so leaf errors propagate and multiply.

5 The Cost Model

5.1 What a cost is

A cost is a single number per plan: a weighted sum of estimated I/O and estimated CPU work, in arbitrary units chosen so that plans can be compared. PostgreSQL's weights, all settable:

Parameter	Default	Charged for
`seq_page_cost`	1.0	one page read in a sequential scan (the unit)
`random_page_cost`	4.0	one page fetched at random (index lookups into the heap)
`cpu_tuple_cost`	0.01	processing one row
`cpu_operator_cost`	0.0025	evaluating one operator or function call (a filter comparison)
`cpu_index_tuple_cost`	0.005	processing one index entry

Chapters 2 to 4 counted only pages because I/O dominates: a page read costs a hundred times a row's CPU work, which is exactly what the 1.0 versus 0.01

ratio says. The CPU terms are there so that two plans with equal I/O can still be told apart, and so that CPU-heavy operations (a regex filter, hashing wide rows) are not priced at zero.

Worked example: Costing a sequential scan with a filter

`orders` has 500,000 rows in 3,550 pages. Cost of `Seq Scan ... Filter: amount > 90` is

$$3550 \times 1.0 + 500{,}000 \times 0.01 + 500{,}000 \times 0.0025 = 3550 + 5000 + 1250 = 9800,$$

which is the `cost=0.00..9800` the lecture's plan prints. The first number, 0.00, is the **startup cost**: a scan emits its first row immediately. A sort would show a large startup cost, because nothing comes out until the whole input is sorted.

5.2 Access path: index or scan

The first physical decision for every table. With selectivity s on a table of N rows and B pages:

- Sequential scan: about B (plus CPU per row), independent of s.
- Index scan: descend the tree (a few pages), then fetch about $s \cdot N$ heap rows, each a *random* page read at cost 4 unless neighbouring matches share a page.

The index wins when $4sN < B$, that is $s < B/(4N) = 1/(4 \cdot \text{rows per page})$. With 141 rows per page that is $s < 0.18\%$. PostgreSQL softens the cliff with the **bitmap heap scan** (Chapter 3: collect matching row ids, sort by page, read each page once, sequentially), which is why in practice the flip from index to scan happens somewhere between 1% and 10% depending on how well the table's physical order follows the column (the `correlation` statistic). Either way, the lesson of Chapter 3 is now a formula: *an index on a predicate that keeps a large fraction of the table is never used*, and the optimizer is right not to use it.

5.3 Costing a whole plan

Mechanism: Pricing a candidate plan

Walk the tree bottom-up. For each operator: take the estimated input sizes from the children (rows and, via rows per page, pages); apply the Chapter 4 formula for that operator with those sizes and the available memory (M from `work_mem`); add the CPU terms. The plan's cost is the sum. Repeat for every candidate; keep the cheapest.

Worked example: The nightly dashboard, priced two ways

Filtered `customers` is believed to be 2,000 rows (15 pages); `orders` is 3,550 pages and has an index on `cust_id`. Index nested loop: $15 + 2000 \times (3 + 50) \approx 106{,}000$ page costs at random-page prices, hash join: $15 + 3550 \approx 3{,}600$. Hash wins. Now suppose the table has grown ten times since `ANALYZE` and the optimizer still believes 2,000 rows while 20,000 are real, but it also still believes `orders` is 3,550 pages while it is 35,500. The believed costs do not change, so the believed winner does not change; the *real* cost of whatever was chosen is ten times the estimate. When the believed outer is small enough, as in the lecture's story ("a few thousand rows"), the nested loop is chosen on paper and then loops over millions in reality: 45 minutes instead of 20 seconds.

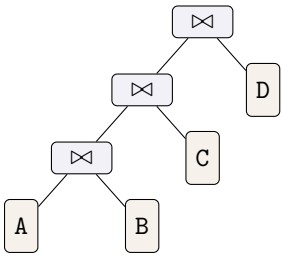
Summary

Cost is a weighted sum of I/O (sequential page 1, random page 4) and CPU (0.01 per row), computed bottom-up from estimated sizes with the Chapter 4 formulas. The access-path rule falls out of it: an index pays only when the predicate keeps a small fraction, roughly below $1/(4 \cdot \text{rows per page})$ before bitmap scans soften it. The cost model is exact arithmetic on inexact inputs.

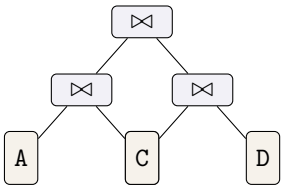
6 Join Ordering

6.1 The search space

With joins commutative and associative, n tables can be joined in any order and any grouping.



left-deep: $n!$ orders



bushy: $(2n-2)!/(n-1)!$ trees

A **left-deep** plan adds one base table per join: the staircase on the left. It is the classic search space because every join's inner side is a base table (so an index on it can be used) and the output of each join streams straight into the next (pipelining, Chapter 4). Even restricted to left-deep plans there are $n!$ orders: $5! = 120$, $10! \approx 3.6$ million, and each order still has a choice of algorithm per join. Trying them all is impossible beyond a handful of tables, and the orders are not interchangeable: joining two large unfiltered tables

first can build a billion-row intermediate result that a different order never materialises.

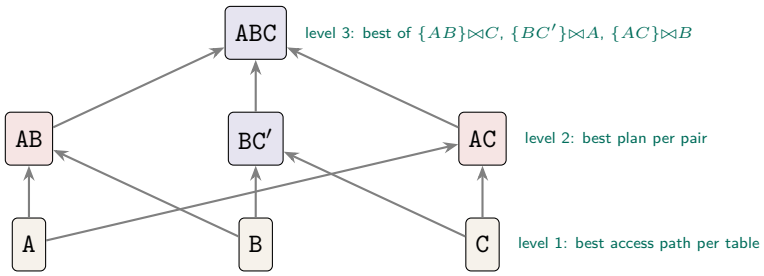
6.2 System-R dynamic programming

Dynamic programming solves a big problem by solving each sub-problem once and reusing the answer. The observation that makes it apply here: *the best plan for joining a set of tables contains the best plan for joining each of its subsets.* If the cheapest way to join $\{A, B, C\}$ ends with $(\ldots) \bowtie C$, then the $(\ldots)$ part must be the cheapest way to join $\{A, B\}$; otherwise replacing it would make the whole cheaper.

Mechanism: System-R join enumeration (Selinger et al., 1979)

1. **Level 1.** For each table, the cheapest access path (scan or each usable index). Keep the best, and also the best plan for each *interesting order* (a sort order on a join or `GROUP BY` column that a later operator could exploit).
2. **Level k.** For each subset of k tables, form it from a best plan of $k-1$ tables joined with one more table, trying every join algorithm; keep the cheapest plan for the subset (and the cheapest per interesting order).
3. **Level n.** The single remaining subset holds the answer.

Subsets, not sequences, are memoised: 2^n of them instead of $n!$ orders. Cross products are considered last or not at all.



Worked example: Running the DP by hand

Tables: A with 1,000 rows (key `id`), B with 10,000 rows and foreign keys `a_id` (1,000 distinct) and `c_id` (100 distinct), C with 100 rows (key `id`). The query joins all three and filters C with a predicate of selectivity 0.1, so $C' = \sigma(C)$ has 10 rows. Use the simplest cost model for a paper exercise: the cost of a plan is the *size of its intermediate result* (the final result is the same size for every order, so it does not discriminate).

- **Level 1.** A: 1,000 rows. B: 10,000. C': 10 (filter pushed down).
- **Level 2.** $A \bowtie B$: $1000 \times 10{,}000 / \max(1000, 1000) = 10{,}000$. $B \bowtie C'$: $10{,}000 \times 10 / \max(100, 10) = 1{,}000$. $A \times C'$: no predicate, cross product, 10,000.
- **Level 3.** $(A \bowtie B) \bowtie C'$: intermediate 10,000. $(B \bowtie C') \bowtie A$: intermediate **1,000**. $(A \times C') \bowtie B$: intermediate 10,000.

Winner: join B with the filtered C' first, then bring in A. The general lesson it illustrates is the one the lecture's six-table story made the hard way: *join the small, selective side first so that every intermediate stays small.* (All sizes verified by the estimation formula of Section 4.)

6.3 Interesting orders, and when DP gives up

An *interesting order* is why level k keeps more than one plan per subset: a sort-merge join on `cust_id` leaves its output sorted on `cust_id`, which a later merge join or a `GROUP BY cust_id` can use without sorting again. A plan that is slightly more expensive but sorted may win at the next level, so it is kept alongside the cheapest unsorted one.

DP is exponential in n: fine for the common case of up to a dozen tables, hopeless beyond. PostgreSQL switches to a **genetic** search (`geqo`) above `geqo_threshold = 12` tables, and caps how many tables it will reorder at all with `join_collapse_limit = 8`: beyond that it keeps the order you wrote. That setting is what the lecture's six-table report tripped over: with a low limit, the optimizer followed the written order, joined two large tables first and built a billion-row intermediate before the selective filter applied. Other systems use greedy heuristics (always pick the cheapest next join) for large n.

History. System R was IBM's 1979 prototype relational system; its access-path selection paper is the origin of cost-based optimization, and every major optimizer still follows its outline.

Summary

Join order is free (commutative, associative) and matters more than anything else in a big join. The $n!$ left-deep orders are searched by System-R dynamic programming: best plan per subset, built level by level from best plans of smaller subsets, keeping extra plans for interesting orders. Beyond a dozen tables, heuristics or genetic search take over, and `join_collapse_limit` can silently freeze the order you wrote.

7 When Plans Go Wrong

7.1 Rule-based versus cost-based

Early optimizers were **rule-based**: a fixed priority list (use an index if one exists; join in written order; prefer nested loop) applied regardless of the data. Simple and predictable, and wrong whenever the data did not fit the rule, such as an index on a predicate that keeps 90% of the rows. Modern optimizers are **cost-based**: they estimate and compare. That is strictly better when the estimates are right, and it introduces a new way to fail: a cost-based optimizer with wrong statistics confidently picks a terrible plan. The three failure modes below are all of that kind.

7.2 Failure 1: stale statistics

Statistics are a snapshot. `ANALYZE` ran at some moment; data changed since; the catalog still describes the old table. Autovacuum re-analyses a table only after roughly 10% of its rows plus 50 have changed, and a bulk load can insert far more than that before it gets around to it.

```
-- add 300,000 orders, do NOT analyze, then:
EXPLAIN ANALYZE SELECT * FROM orders WHERE amount > 90;    -- stale
ANALYZE orders;
EXPLAIN ANALYZE SELECT * FROM orders WHERE amount > 90;    -- fresh
```

```
before:  rows=4873   actual rows=7800    <- estimate far too low
after:   rows=7790   actual rows=7800    <- matches again
```

Before `ANALYZE` the planner still believes the 500,000-row table and scales its selectivity by that count; the inserts raised the real count by 60%, so every estimate is 60% low. One `ANALYZE` rebuilds the row count and the histogram, and the estimates snap back. In the lecture's real-world version the table had grown 10×, the join's outer side was believed tiny, a nested loop was chosen, and the dashboard took 45 minutes; after one `ANALYZE` it took 20 seconds. Stale statistics are the first thing to suspect and the cheapest thing to fix.

7.3 Failure 2: correlated columns

The independence assumption multiplies selectivities. When one column determines another, the product is wrong by exactly the strength of the dependency.

```
EXPLAIN ANALYZE SELECT * FROM customers WHERE country = 'MA' AND city
     = 'Casablanca';
CREATE STATISTICS s_geo (dependencies) ON country, city FROM
    customers;
ANALYZE customers;    -- then re-run the EXPLAIN
```

```
before:  rows=400    actual rows=2000    <- 0.2 x 0.2 x 10000 = 400
after:   rows=1950   actual rows=2000    <- dependency now known
```

Every Casablanca customer is Moroccan, so `city = 'Casablanca'` already implies `country = 'MA'` and the conjunction keeps $0.2 \times 10{,}000 = 2{,}000$ rows, not 400. The estimate is 5× too low; with a stronger dependency or a third correlated predicate it would be 25× or 125×. **Extended statistics** (`CREATE STATISTICS ... (dependencies)`) measure how strongly `city` determines `country` and the planner stops multiplying. The other kinds, `ndistinct` (joint distinct counts for `GROUP BY` on several columns) and `mcv` (joint most-common combinations), fix the same assumption in other places.

7.4 Failure 3: skew

If one value dominates a column, "selectivity = 1/n_distinct" is wrong in both directions: too low for the heavy value, too high for every other. The MCV list exists for this: the top values are stored with their exact frequencies, and the remaining rows are spread uniformly over the remaining distinct values. Skew hurts when the heavy value is not in the list (raise `default_statistics_target` or `ALTER TABLE ... SET STATISTICS` on that column) or when the predicate's constant is a parameter the planner cannot see (`col = $1`): then it must use the average, which is wrong for the heavy value. A plan that is perfect for `event_type = 'purchase'` (0.5% of rows, index) is a disaster for `event_type = 'view'` (90%, scan), and a prepared statement may be forced to use one plan for both.

7.5 Reading EXPLAIN like a diagnostician

Concept: The anatomy of a plan line

```
Seq Scan on orders  (cost=0.00..9800 rows=5000 width=32) (actual time=0.01..61.2
    rows=4873 loops=1)
  Filter: (amount > 90)
  Rows Removed by Filter: 495127
  Buffers: shared hit=3550
```

`cost=startup..total`: the optimizer's estimate, in cost units, before and after producing all rows. `rows`: estimated output rows. `width`: estimated bytes per row (projection pushdown made it small). `actual time=first..last` in milliseconds and `actual rows`: measured, shown only by `EXPLAIN ANALYZE`, which really runs the query. `loops`: how many times the node ran; for the inner side of a nested loop it is the outer's row count, and the `actual rows` shown is *per loop*. `Rows Removed by Filter`: rows read and thrown away, wasted work. `Buffers`: pages found in the pool (`hit`) versus fetched from disk (`read`), with the `BUFFERS` option.

Technique: Diagnosing a slow plan

1. Run `EXPLAIN (ANALYZE, BUFFERS)` and compare `rows` with `actual rows` at every node, starting from the leaves. A ratio above 10× either way is the problem; find the *lowest* node where it first appears, since everything above inherits it.
2. Ask why the leaf is wrong. Single predicate, large table recently loaded: stale statistics, run `ANALYZE`. Several predicates on related columns: correlation, `CREATE STATISTICS`. A predicate on a lopsided column: skew, raise the statistics target.
3. Look at what the wrong estimate caused: a `Nested Loop` with a huge `loops` count on its inner side, a `Hash` with `Batches` greater than 1 (the build side spilled), a `Sort Method: external merge`. Those are the symptoms; the estimate is the disease.

4. Fix the statistics, re-run, and confirm the plan changed (typically to a hash join) and the `actual time` dropped. Only if the estimate is now right and the plan is still bad, look at indexes, `work_mem`, or the query itself.

Worked example: Pushdown and join choice, seen in one real plan

```
HashAggregate
  -> Hash Join  (Hash Cond: o.cust_id = c.id)
       -> Seq Scan on orders o
       -> Hash
            -> Seq Scan on customers c
                 Filter: (country = 'MA')        <- pushed below the join
```

Logical and physical optimization in one picture. The filter sits on the customers scan, below the join (selection pushdown). The filtered customers, about 2,000 rows, became the *build* side of a hash join (commutativity: the optimizer chose which side to hash, and it picked the small one), and the 500,000 orders are the probe side, read once. Had the estimate for the filter been 200,000 instead of 2,000, the build side might not have fit `work_mem`, and the optimizer might have preferred a merge join or a multi-batch hash.

Summary

Cost-based optimizers fail through their inputs: stale statistics (fix: `ANALYZE`), correlated columns (fix: `CREATE STATISTICS`), and skew (fix: a larger MCV list). Diagnose from `EXPLAIN ANALYZE` by comparing estimated and actual rows, lowest wrong node first; the wrong join algorithm or spilled hash is the symptom, the estimate is the cause.

In the real world. PostgreSQL has no optimizer hints by design. The levers are statistics (`ANALYZE`, `CREATE STATISTICS`, statistics targets), cost parameters (`random_page_cost` is often lowered to 1.1 on SSDs), memory (`work_mem`), the `enable_*` switches for experiments, and, in extremity, the `pg_hint_plan` extension.

8 Lab 5 Companion

Lab 5 reproduces this chapter's two failures on `adm_lab` and asks you to measure the estimates.

Technique: Part by part

1. **Estimated vs. actual.** For the single-table filter expect `rows` and `actual rows` within a few percent of each other (fresh statistics from the earlier labs). For the join, read bottom-up: the customers scan with its `Filter`, the `Hash` on it, the orders scan, the `Hash Join`. Record the pair at each node; the join's actual rows should be about the filtered customers' share of orders.
2. **Read the statistics.** `pg_stats` for `orders`: `cust_id` with a large `n_distinct`, `amount` with no MCVs and a histogram, `order_date` likewise. Note `null_frac = 0` everywhere.
3. **Selectivity by hand.** The `count(*) FILTER` query gives the true

fraction of Moroccan customers, about 0.2 if the generator was uniform over five countries. Multiply by the row count and compare with the `rows=` of the `EXPLAIN`: they should agree closely, because `country` has only five distinct values and all five are in the MCV list with their exact frequencies.

4. **Stale statistics.** The insert adds 300,000 orders. Before `ANALYZE`, the estimate for `amount > 90` is still scaled to the old row count, so it is about 60% of the actual. After `ANALYZE`, it matches. Report both numbers and the ratio.
5. **Correlated columns.** After adding `city` as a function of `country`, the conjunction estimate is $0.2 \times 0.2 \times 20{,}000 = 800$ against about 4,000 actual (your table has 20,000 customers; the deck's 10,000 gives 400 against 2,000). After `CREATE STATISTICS s_geo (dependencies)` and `ANALYZE`, the estimate moves to within a few percent of the actual.
6. **The short answer.** A wrong estimate at a leaf changes the estimated size of every operator above it, and therefore the choice of join algorithm and join order; a wrong estimate at the top only misprices the last operator, whose algorithm is already chosen with the right inputs.

Debugging: If the experiment does not show the effect

- **Estimates already accurate before `ANALYZE`.** Autovacuum ran in between. Insert more rows, or disable it for the experiment: `ALTER TABLE orders SET (autovacuum_enabled = false);` and re-enable afterwards.
- **`CREATE STATISTICS` changes little.** The dependency must be strong; the lab's `CASE` makes `city` a function of `country`, which is as strong as it gets. Check that you ran `ANALYZE` *after* creating the statistics object; it is empty until then.
- **The plan does not change even though the estimate did.** It need not: a 5× error on a single-table scan still leaves "sequential scan" the right choice. The estimate is the deliverable here; the plan flip appears in multi-table queries such as the lecture's dashboard.
- **`pg_stats` shows no row for a column.** It has never been analysed. `ANALYZE` the table.

9 Tricks, Techniques and Classic Problems

Warning / Gotcha: Common student mistakes

- **Adding selectivities for AND.** $\text{sel}(p \wedge q) = \text{sel}(p)\,\text{sel}(q)$ under independence; the sum is wrong, and even the product is wrong when the columns are correlated.
- **Dividing by $d_R + d_S$ or by** min **in the join formula.** It is $\max(d_R, d_S)$; for a key–foreign key join the answer is simply the size of

the foreign-key side.

- **Saying the independence error is "too high".** For correlated predicates in an AND the product *under*-estimates; the true count is larger.
- **Counting $n!$ as the number of plans.** It is the number of left-deep *orders*; bushy trees, join algorithms and access paths multiply it further. DP memoises 2^n subsets, not $n!$ orders.
- **Fixing a bad plan by adding an index.** If the estimate is wrong, the optimizer may not use the index, or may use it when it should not. Fix the statistics first, then revisit indexes.
- **Reading `actual rows` on a nested-loop inner side as a total.** It is per loop; multiply by `loops`.
- **Treating `cost` as time.** It is in arbitrary units (sequential page reads); only compare costs of plans for the same query.

Worked example: Classic exam and interview problems

P1. A 2,000,000-row table; `country = 'MA'` has selectivity 0.2, `city = 'Casablanca'` 0.1, and every Casablanca customer is in MA. Independence estimate and true count for the AND?
Independence: $0.2 \times 0.1 \times 2{,}000{,}000 = 40{,}000$. True: city implies country, so the AND is just the city predicate: $0.1 \times 2{,}000{,}000 = 200{,}000$. Five times too low; fix with `CREATE STATISTICS (dependencies) ON country, city` then `ANALYZE`.

P2. `customers` (10,000 rows, key `id`) joined to `orders` (500,000 rows, `cust_id` with 10,000 distinct values). Estimated join size? And after `country = 'MA'` (sel 0.2) on customers?
$10{,}000 \times 500{,}000 / \max(10{,}000, 10{,}000) = 500{,}000$: every order matches its one customer. After the filter: $2{,}000 \times 500{,}000/10{,}000 = 100{,}000$.

P3. A node shows `rows=50` and `actual rows=500000`. Two causes and the first command?
Stale statistics after many inserts, or correlated predicates multiplied as if independent. First run `ANALYZE` on the table; if the gap persists and the filter has several columns, `CREATE STATISTICS` on them and analyse again.

P4. Why is cardinality estimation the hardest and most important job of the optimizer?
Because every later decision is a function of estimated row counts: join algorithm (hash needs the build side to fit), join order (small intermediates first), access path (index only below a selectivity threshold). Rewriting is mechanical and always safe, costing is arithmetic; predicting how many rows a skewed, correlated, changing dataset will produce is the genuinely uncertain step, and an error at a leaf is multiplied at every join above it.

P5. Four tables, left-deep only. How many orders? How many subsets does DP evaluate?
$4! = 24$ orders; DP keeps one best plan (plus interesting orders) for each of $2^4 - 1 = 15$ non-empty subsets and builds level k from level $k - 1$.

P6. A query joins six tables in written order and runs out of memory. What setting is involved and what does the fix look like?

`join_collapse_limit` (default 8) caps how many tables are reordered; written with explicit `JOIN` syntax and a low limit, the optimizer keeps the written order and may join two big tables before the selective filter applies. Raise the limit or rewrite so the selective join comes first; every intermediate then stays small.

P7. Why would the planner ignore a B-tree index on a column where the query keeps 40% of the rows?

Because $0.4 \times N$ random heap fetches at cost 4 each exceed one sequential pass over B pages by a wide margin (the index wins only below about $1/(4 \cdot \text{rows per page})$, before bitmap scans); the sequential scan is correct, and the index costs writes and space for nothing.

P8. `EXPLAIN` versus `EXPLAIN ANALYZE`: which one would you never run on a production `DELETE`?

`EXPLAIN ANALYZE` actually executes the statement. Wrap it in a transaction and roll back, or use plain `EXPLAIN`.

Summary: Chapter 5 in six lines

1. The optimizer turns a logical plan into the cheapest physical plan it can find, without running anything, from catalog statistics.
2. Logical rewrites (selection and projection pushdown, subquery flattening, constant folding) are always answer-preserving; join commutativity and associativity make the order free.
3. Selectivity is a probability: 1/n_distinct or MCV frequency for equality, histogram area for ranges, product for AND under independence; a join is $|R||S|/\max(d_R, d_S)$.
4. Cost is a weighted sum of I/O and CPU computed bottom-up with the Chapter 4 formulas on estimated sizes; a wrong leaf estimate is multiplied at every join above.
5. Join order is searched by System-R dynamic programming over subsets, keeping interesting orders; large joins fall back to heuristics, and `join_collapse_limit` can freeze the written order.
6. Most bad plans are bad estimates: stale statistics (`ANALYZE`), correlation (`CREATE STATISTICS`), skew (bigger MCV list). Read `EXPLAIN ANALYZE` from the leaves up, estimated versus actual.